

CHAPTER VI

Pig A High Level Data Processing System on Hadoop

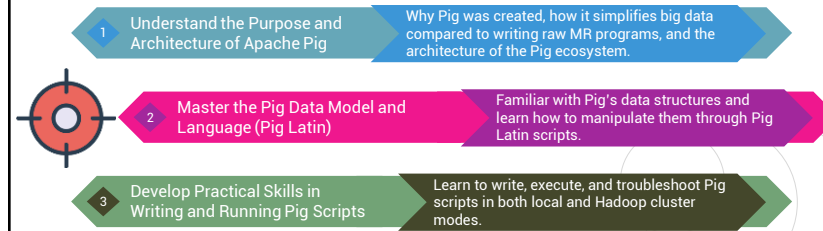
1. INTRODUCTION TO PIG
2. PIG LATIN BASICS
3. PIG SCRIPTING
4. PIG FUNCTIONS
5. USER DEFINED FUNCTIONS

Yet Another Resource Negotiator

Objectives

Section 01

2

Fouad DAHAK
ESIA, 2024/2025

ensia

Section N° 1

Introduction to PIG

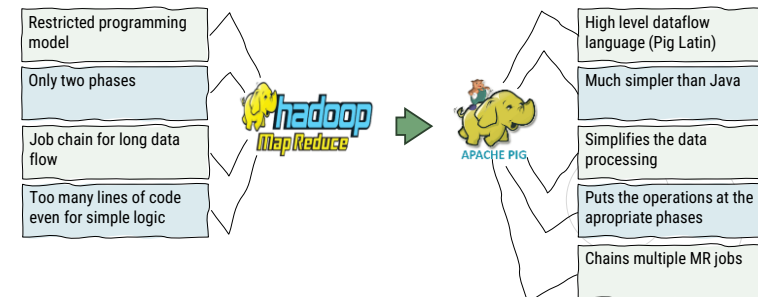
Fouad DAHAK, ENSIA, 2024/2025

Introduction to PIG

Is Map Reduce not enough?

Section 01

4

Fouad DAHAK
ESIA, 2024/2025

ensia

Introduction to PIG

What is Pig?

Pig Latin is a data flow language used for exploring large data sets.

Rapid development No Java is required.

It's a high-level platform for creating MapReduce programs used with Hadoop.

Developed at Yahoo around 2006
In 2007, it was moved into the Apache Software Foundation

Designed for performing a long series of data operations, making it ideal for three categories of Big Data jobs:

- Extract-transform-load (ETL) data pipelines,
- Research on raw data
- Iterative data processing.

At Yahoo, 70% MR jobs are written in Pig :
Process web logs : Build user behavior models. Also used by Twitter, LinkedIn, eBay, AOL, ...

APACHE PIG

Fouad DAHAK
ESIA, 2024/2025

Introduction to PIG

Pig Modes

Local Mode

Running the PIG script on a local machine. We need to install HADOOP, PIG, JAVA on a local machine. All files are installed and run using the local host and file system. Used for local testing.

`pig -x local script_name.pig`

Mapreduce Mode

We need access to the hadoop cluster and HDFS installations. Most used for production runs.

`pig -x mapreduce script_name.pig`

Spark Mode

We need access to hadoop cluster, Yarn, Spark and HDFS.

`pig -x spark script_name.pig`

Tez Mode

Similar to mapreduce mode but this invokes the TEZ engine instead of the MR execution engine. We need access to Hadoop cluster and HDFS.

`pig -x tez script_name.pig`

APACHE PIG

Fouad DAHAK
ESIA, 2024/2025

Introduction to PIG

Architecture of Pig

Pig converts the scripts into a series of MapReduce jobs

Pig Latin Script

Grunt Shell | Pig Server

Parser

Optimizer

Compiler

Execution Engine

Map Reduce

Hadoop

HDFS

Parser: Checks the syntax of the script, does type checking. The output will be a DAG, which represents the Pig Latin statements and logical operators.

Optimizer: The logical plan (DAG) is passed to the logical optimizer, which carries out the logical optimizations

Compiler: Compiles the optimized logical plan into a series of MapReduce jobs

Execution engine: MapReduce jobs are submitted to Hadoop in a sorted order. These MapReduce jobs are executed on Hadoop producing the desired results.

APACHE PIG

Fouad DAHAK
ESIA, 2024/2025

Introduction to PIG

Data Model

The data model of Pig Latin is fully nested and it allows complex non-atomic datatypes such as map and tuple.

Atom	Field	Tuple	Bag	Map	Relation
Any single value irrespective of their data type. It is stored as string and can be used as string and number.	A piece of data or a simple atomic value.	A record that is formed by an ordered set of fields, the fields can be of any type. A tuple is similar to a row in a table of RDBMS.	A bag is an unordered set of tuples. Each tuple can have any number of fields. Represented by {}.	A map is a set of key-value pairs. The key needs to be of type chararray and should be unique. The value might be of any type. It is represented by []	A relation is a bag of tuples. The relations in Pig Latin are unordered (there is no guarantee that tuples are processed in any particular order).

{
(Ahmad, 22, {(Math, 18), (Physics, 17), (History, 12)})
(Malik, 21, {(Math, 10), (Physics, 14), (History, 19)})
}

APACHE PIG

Fouad DAHAK
ESIA, 2024/2025

Introduction to PIG

Pig Grunt Shell Commands

Grunt shell is a shell command. It is mainly used to write pig Latin scripts. Pig script can be executed with grunt shell which is native shell provided by Apache pig to execute pig queries.

Command	Description	Example
FS	Executes Hadoop File System (HDFS) commands directly within the Grunt shell. This allows you to manage files and directories on HDFS without leaving Pig.	FS -ls /user/hadoop/
SH	Executes a local shell command from within the Grunt shell. It allows you to run commands on the operating system where Pig is running.	SH ls -l /home/user/

Command	Description	Example
CLEAR	Clears all the aliases (relations) currently defined in the session. Like resetting the environment without quitting.	CLEAR;
EXEC	Runs a Pig script file immediately. It starts execution right away (no Grunt session interaction).	EXEC /home/user/scripts/myscript.pig;
HELP	Displays help information about Pig commands and usage.	HELP;
HISTORY	Displays the command history for the current Grunt session. Useful to review past commands.	HISTORY;
KILL	Terminates a running MapReduce job started by Pig. You need the Job ID.	KILL job_202504280123_0005;
QUIT	Exits the Pig Grunt shell.	QUIT;
RUN	Runs a Pig script file but keeps the Grunt session active after the script runs.	RUN /home/user/scripts/myscript.pig;
SET	Changes a Pig or Hadoop property for the current session.	SET default_parallel 5;

Shell Commands

Utility Commands

fs
sh

Clear
Exec
help
history
kill
quit
run
set

Fouad DAHAK
ESIA, 2024/2025

Section N° 2

Pig Latin Basics

Fouad DAHAK, ENSIA, 2024/2025

Pig Latin Basics

Pig Latin Statements

While processing data using Pig Latin, statements are the basic constructs.

These statements work with relations. They include expressions and schemas..

Every statement ends with a semicolon (;). d out.

We will perform various operations using operators provided by Pig Latin, through statements..

Except LOAD and STORE, while performing all other operations, Pig Latin statements take a relation as input and produce another relation as output.

As soon as you enter a Load statement in the Grunt shell, its semantic checking will be carried out. To see the contents of the schema, you need to use the Dump operator. Only after performing the dump operation, the MapReduce job for loading the data into the file system will be carried out.

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Pig Latin Data types

Data Type	Description & Example
int	Represents a signed 32-bit integer.
long	Represents a signed 64-bit integer.
float	Represents a signed 32-bit floating point.
double	Represents a 64-bit floating point.
chararray	Represents a character array (string) in Unicode UTF-8 format.
Bytearray	Represents a Byte array (blob).
Boolean	Represents a Boolean value.
Datetime	Represents a date-time.
Biginteger	Represents a Java BigInteger.
Bigdecimal	Represents a Java BigDecimal

Complex Types
Tuple A tuple is an ordered set of fields. Example : (Anis, 17)
Bag A bag is a collection of tuples. Example : {(Anis,17),(Yacine,15)}
Map A Map is a set of key-value pairs. Example : [name#Anis, age#17]

Null Values
Apache Pig treats null values in a similar way as SQL does. A null can be an unknown value or a non-existent value.

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Pig Latin Arithmetic and Comparison Operators

Section 02

13

Pig Latin Arithmetic Operators

Operator	Description	Example
+	Addition	a + b will give 30
-	Subtraction	a - b will give -10
*	Multiplication	a * b will give 200
/	Division	b / a will give 2
%	Modulus	b % a will give 0
?:	Bincond - Evaluates the Boolean operators. It has three operands as shown below. variable x = (expression) ? value1 if true : value2 if false.	b = (a == 1)? 20: 30;
CASE	Case - The case operator is equivalent to nested bincond operator.	CASE f2 % 2 WHEN 0 THEN 'even' WHEN 1 THEN 'odd' END

Pig Latin Comparison Operators

Operator	Description	Example
==	Equal	(a = b) is not true
!=	Not Equal	(a != b) is true.
>	Greater than	(a > b) is not true.
<	Less than	(a < b) is true.
>=	Greater than or equal to	(a >= b) is not true.
<=	Less than or equal to	(a <= b) is true.
matches	Pattern matching	f1 matches '.*tutorial.*'

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Pig Latin Relational Operations

Section 02

14

Category	Operator	Description
Loading and Storing	LOAD	To Load the data from the file system into a relation.
	STORE	To save a relation to the file system (local/HDFS).
Filtering	FILTER	To remove unwanted rows from a relation.
	DISTINCT	To remove duplicate rows from a relation.
	FOREACH, GENERATE	To generate data transformations based on columns of data.
Grouping and Joining	STREAM	To transform a relation using an external program.
	JOIN	To join two or more relations.
	COGROUP	To group the data in two or more relations.
	GROUP	To group the data in a single relation.
	CROSS	To create the cross product of two or more relations.
Sorting	ORDER	Arrange a relation in a sorted order.
	LIMIT	To get a limited number of tuples from a relation.
Combining and Splitting	UNION	To combine two or more relations into a single relation.
	SPLIT	To split a single relation into two or more relations.

Diagnostic Operators

Operator	Description
DUMP	To print the contents of a relation on the console.
DESCRIBE	To describe the schema of a relation.
EXPLAIN	To view the logical, physical, or MapReduce execution plans to compute a relation.
ILLUSTRATE	To view the step-by-step execution of a series of statements.

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

The Load Operator

Section 02

15

Relation_name = LOAD 'Input file path' USING function as schema;

- relation_name - We have to mention the relation in which we want to store the data.
- Input file path - We have to mention the HDFS directory where the file is stored. (In MapReduce mode)
- function - We have to choose a function from the set of load functions provided by Apache Pig (BinStorage, JsonLoader, PigStorage, TextLoader).
- Schema - We have to define the schema of the data. We can define the required schema as follows-

```
Contributions_Raw = LOAD
'/shared/employee_contributions.csv'
USING PigStorage(',')
AS (year:int, month:int, employee_id:chararray,
contribution:double, gender:chararray, code1:chararray,
code2:chararray, misc:int);
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Storing Data

Section 02

16

STORE Relation_name INTO 'required_directory_path' [USING function];

You can store the loaded data in the file system using the store operator.

```
STORE FEmployees INTO '/output/female_employees'
USING PigStorage(',');
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Diagnostic Operators

Operator Relation_name;

- ☐ Dump operator
- ☐ Describe operator
- ☐ Explanation operator
- ☐ Illustration operator

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Group Operator

Group_data = GROUP Relation_name BY age;

The **GROUP** operator is used to group the data in one or more relations. It collects the data having the same key.

Grouping by Multiple Columns
Let us group the relation by age and city as shown below.

Group All
You can group a relation by all the columns as shown below.

Relation A (employees):

name	department
John	IT
Sarah	HR
Ahmed	IT
Maria	HR
Ali	Finance

B = GROUP A BY department;
DUMP B;

(IT, ((John,IT), (Ahmed,IT)))
(HR, ((Sarah,HR), (Maria,HR)))
(Finance, ((Ali,Finance)))

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Cogroup Operator

cogroup_data = COGROUP student_details by age, employee_details by age;

The **COGROUP** operator works more or less in the same way as the **GROUP** operator. The only difference between the two operators is that the group operator is normally used with one relation, while the cogroup operator is used in statements involving two or more relations.

Relation A (employees):

emp_id	name
1	John
2	Sarah
3	Ahmed

Relation B (salaries):

emp_id	salary
1	4000
2	3500
4	5000

C = COGROUP A BY emp_id, B BY emp_id;
DUMP C;

(1, ((1,John)), ((1,4000)))
(2, ((2,Sarah)), ((2,3500)))
(3, ((3,Ahmed)), {}) -- No salary for Ahmed
(4, {}, ((4,5000))) -- No employee with id 4

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Join Operator

Relation3_name = JOIN Relation1_name BY key [LEFT|RIGHT|FULL OUTER], Relation2_name BY key ;

The **JOIN** operator is used to combine records from two or more relations. While performing a join operation, we declare one (or a group of) tuple(s) from each relation, as keys. When these keys match, the two particular tuples are matched, else the records are dropped. Joins can be of the following types -

- ☐ Self-join
- ☐ Inner-join
- ☐ Outer-join - left join, right join, and full join

Relation A (employees):

emp_id	name
1	John
2	Sarah
3	Ahmed

Relation B (salaries):

emp_id	salary
1	4000
2	3500
4	5000

C = JOIN A BY emp_id, B BY emp_id;
DUMP C;

(1,John,1,4000)
(2,Sarah,2,3500)

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Cross Operator

Relation3_name = **CROSS** Relation1_name, Relation2_name;

The CROSS operator computes the cross-product of two or more relations. This chapter explains with example how to use the cross operator in Pig Latin.

```
employees = LOAD '/shared/employees.csv'
USING PigStorage(',')
AS (id:chararray, name:chararray);

departments = LOAD '/shared/departments.csv'
USING PigStorage(',')
AS (dept_name:chararray);

cross_result = CROSS employees, departments;

DUMP cross_result;
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Union Operator

Relation3_name = **UNION** Relation1_name, Relation2_name;

The UNION operator of Pig Latin is used to merge the content of two relations. To perform UNION operation on two relations, their columns and domains must be identical.

```
dept1 = LOAD '/shared/employees_dept1.csv'
USING PigStorage(',')
AS (id:chararray, name:chararray, gender:chararray);

dept2 = LOAD '/shared/employees_dept2.csv'
USING PigStorage(',')
AS (id:chararray, name:chararray, gender:chararray);

all_employees = UNION dept1, dept2;

DUMP all_employees;
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Split Operator

SPLIT Relation1_name INTO Relation2_name IF (condition1), Relation2_name IF (condition2);

The SPLIT operator is used to split a relation into two or more relations.

```
employees = LOAD '/shared/employees.csv'
USING PigStorage(',')
AS (id:chararray, name:chararray, gender:chararray);

SPLIT employees INTO
males IF gender == 'MALE',
females IF gender == 'FEMALE';

DUMP males;
DUMP females;
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Filter Operator

Relation2_name = **FILTER** Relation1_name BY (condition);

The FILTER operator is used to select the required tuples from a relation based on a condition.

```
A = LOAD '/shared/employees.csv' USING PigStorage(',')
AS (employee_id:chararray, employee_name:chararray,
gender:chararray);

B = FILTER A BY gender == 'MALE';
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Distinct Operator

Relation2_name = **DISTINCT** Relation1_name;

The **DISTINCT** operator is used to remove redundant (duplicate) tuples from a relation.

```
employees = LOAD '/shared/employees.csv'
USING PigStorage(',')
AS (id:chararray, name:chararray, gender:chararray);

names = FOREACH employees GENERATE name, gender;

unique_names = DISTINCT names;

DUMP unique_names;
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Foreach Operator

Relation2_name = **FOREACH** Relatin_name1 **GENERATE** (required data);

The **FOREACH** operator is used to generate specified data transformations based on the column data.

```
B = FOREACH A GENERATE employee_name, gender;
DUMP B;
```

```
(THELD,JA,FEMALE)
(NOUREDINE,MALE)
(FAIZA,FEMALE)
(HAKIM,MALE)
(MOURAD,MALE)
(REDOUANE,MALE)
(BENYOUCEF,MALE)
(ASMA,FEMALE)
(SLIMANE,MALE)
(NOUREDINE,MALE)
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Order By Operator

Relation2_name = **ORDER** Relatin_name1 **BY** Field (ASC|DESC);

The **ORDER BY** operator is used to display the contents of a relation in a sorted order based on one or more fields.

```
D = ORDER B BY employee_name;
DUMP D;
```

```
(THELD,JA,FEMALE)
(NOUREDINE,MALE)
(FAIZA,FEMALE)
(HAKIM,MALE)
(MOURAD,MALE)
(REDOUANE,MALE)
(BENYOUCEF,MALE)
(ASMA,FEMALE)
(SLIMANE,MALE)
(NOUREDINE,MALE)
```

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Limit Operator

Relation2_name = **LIMIT** Relatin_name1 Number_Of_Tuples;

The **LIMIT** operator is used to get a limited number of tuples from a relation.

```
employees = LOAD '/shared/employees.csv'
USING PigStorage(',')
AS (id:chararray, name:chararray, gender:chararray);

limited_employees = LIMIT employees 3;

DUMP limited_employees;
```

Fouad DAHAK
ESIA, 2024/2025

Section N° 3

Pig Scripting

Fouad DAHAK, ENSIA, 2024/2025

Pig Latin Basics

Examples

"For each employee, calculate the total amount of his/her contributions."

```
contributions_raw = LOAD '/shared/employee_contributions.csv'
USING PigStorage(',')
AS (year:int, month:int, id:chararray, amount:double,
gender:chararray, department_id:chararray, wilaya_id:chararray,
contrib_type:int);

grouped_contributions = GROUP contributions_raw BY id;

total_contributions = FOREACH grouped_contributions GENERATE
group AS employee_id,
SUM(contributions_raw.amount) AS total_amount;

DUMP total_contributions;
```



Fouad DAHAK, ENSIA, 2024/2025

ensia

Pig Latin Basics

Examples

"Give the total contributions per year."

```
contributions_raw = LOAD '/shared/employee_contributions.csv'
USING PigStorage(',')
AS (year:int, month:int, id:chararray, amount:double,
gender:chararray, department_id:chararray, wilaya_id:chararray,
contrib_type:int);

grouped_by_year = GROUP contributions_raw BY year;

total_per_year = FOREACH grouped_by_year GENERATE
group AS year,
SUM(contributions_raw.amount) AS total_amount;

ordered_total_per_year = ORDER total_per_year BY year ASC;

DUMP ordered_total_per_year;
```



Fouad DAHAK, ENSIA, 2024/2025

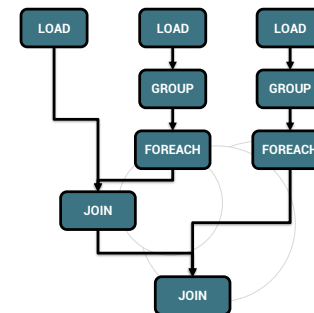
ensia

Pig Latin Basics

Examples

For each premium category, determine the total amount reimbursed.

```
MP = LOAD 'employees.csv' USING PigStorage(',')
AS (id:chararray, name:chararray, gender:chararray);
CR = LOAD 'employee_contributions.csv' USING PigStorage(',')
AS (year:int, month:int, id:chararray, amount:double,
gender:chararray, department_id:chararray, wilaya_id:chararray,
contrib_type:int);
RR = LOAD 'employee_reimbursements.csv' USING PigStorage(',')
AS (employee_id:chararray, reimbursement_id:chararray,
reimbursement_date:chararray, amount:double);
A = GROUP CR BY id;
B = FOREACH A GENERATE group AS id, SUM(CR.amount) AS B;
C = GROUP RR BY employee_id;
D = FOREACH C GENERATE group AS id, SUM(RR.amount) AS D;
E = LEFT OUTER JOIN MP BY id, B BY id;
F = LEFT OUTER JOIN E BY MP::id, D BY id;
G = FOREACH F GENERATE F::MP::id AS employee_id,
F::MP::name AS name, F::MP::gender AS gender,
(F::B IS NULL ? 0.0 : F::B) AS total_contribution,
(D::D IS NULL ? 0.0 : D::D) AS total_reimbursement;
```



Fouad DAHAK, ENSIA, 2024/2025

ensia

Section N° 4

Functions

Fouad DAHAK, ENSIA, 2024/2025

Pig Latin Basics

Eval Functions

Apache Pig provides various built-in functions namely eval, load, store, math, string, bag and tuple functions.

Function	Description
AVG()	To compute the average of the numerical values within a bag.
BagToString()	To concatenate the elements of a bag into a string. While concatenating, we can place a delimiter between these values.
CONCAT()	To concatenate two or more expressions of same type.
COUNT()	To get the number of elements in a bag, while counting the number of tuples in a bag.
COUNT_STAR()	It is similar to the COUNT() function. It is used to get the number of elements in a bag.
DIFF()	To compare two bags (fields) in a tuple.
IsEmpty()	To check if a bag or map is empty.
MAX()	To calculate the highest value for a column (numeric values or chararrays) in a single-column bag.
MIN()	To get the minimum (lowest) value (numeric or chararray) for a certain column in a single-column bag.
PluckTuple()	To define a string Prefix and filter the columns in a relation that begin with the given prefix.
SIZE()	To compute the number of elements based on any Pig data type.
SUBTRACT()	To subtract two bags. It returns a bag which contains the tuples of the first bag that are not in the second bag.
SUM()	To get the total of the numeric values of a column in a single-column bag.
TOKENIZE()	To split a string in a single tuple and return a bag which contains the output of the split operation.

ESIA, 2024/2025 ENSIA

Pig Latin Basics

Bag & Tuple Functions

Function	Description
TOBAG()	To convert two or more expressions into a bag.
TOP()	To get the top N tuples of a relation.
TOTUPLE()	To convert one or more expressions into a tuple.
TOMAP()	To convert the key-value pairs into a Map.

Fouad DAHAK
ESIA, 2024/2025 ENSIA

Pig Latin Basics

String Functions

Functions	Description
ENDSWITH(string, testAgainst)	To verify whether a given string ends with a particular substring.
STARTSWITH(string, substring)	Accepts two string parameters and verifies whether the first string starts with the second.
SUBSTRING(string, startIndex, stopIndex)	Returns a substring from a given string.
EqualsIgnoreCase(string1, string2)	To compare two strings ignoring the case.
INDEXOF(string, character, startIndex)	Returns the first occurrence of a character in a string, searching forward from a start index.
LAST_INDEX_OF(expression)	Returns the index of the last occurrence of a character in a string, searching backward from a start index.
LCFIRST(expression)	Converts the first character in a string to lower case.
UCFIRST(expression)	Returns a string with the first character converted to upper case.
UPPER(expression)	UPPER(expression) Returns a string converted to upper case.
LOWER(expression)	Converts all characters in a string to lower case.
REPLACE(string, oldChar, newChar)	To replace existing characters in a string with new characters.
STRSPLIT(string, regex, limit)	To split a string around matches of a given regular expression.
STRSPLITTOBAG(string, regex, limit)	Similar to the STRSPLIT() function, it splits the string by given delimiter and returns the result in a bag.
TRIM(expression)	Returns a copy of a string with leading and trailing whitespaces removed.
LTRIM(expression)	Returns a copy of a string with leading whitespaces removed.
RTRIM(expression)	Returns a copy of a string with trailing whitespaces removed.

ESIA, 2024/2025 ENSIA

Pig Latin Basics

DateTime Functions

Section 04 37

Functions	Description
ToDate(milliseconds)	Returns a date-time object according to the given parameters.
currentTime()	returns the date-time object of the current time.
GetDay(datetime)	Returns the day of a month from the date-time object.
GetHour(datetime)	Returns the hour of a day from the date-time object.
GetMilliSecond(datetime)	Returns the millisecond of a second from the date-time object.
GetMinute(datetime)	Returns the minute of an hour from the date-time object.
GetMonth(datetime)	Returns the month of a year from the date-time object.
GetSecond(datetime)	Returns the second of a minute from the date-time object.
GetWeek(datetime)	Returns the week of a year from the date-time object.
GetWeekYear(datetime)	Returns the week year from the date-time object.
GetYear(datetime)	Returns the year from the date-time object.
AddDuration(datetime, duration)	Returns the result of a date-time object along with the duration object.
SubtractDuration(datetime, duration)	Subtracts the Duration object from the Date-Time object and returns the result.
DaysBetween HoursBetween MilliSecondsBetween MinutesBetween MonthsBetween SecondsBetween WeeksBetween YearsBetween(d1, d2)	Returns the number of Days Hours Milliseconds Minutes Months Seconds Weeks Years between the two date-time objects.

Fouad DAHAK
ESIA, 2024/2025

Pig Latin Basics

Math Functions

Section 04 38

Functions & Description	Functions & Description
ABS(expression)	To get the absolute value of an expression.
ACOS(expression)	To get the arc cosine of an expression.
ASIN(expression)	To get the arc sine of an expression.
ATAN(expression)	This function is used to get the arc tangent of an expression.
CBRT(expression)	This function is used to get the cube root of an expression.
CEIL(expression)	This function is used to get the value of an expression rounded up to the nearest integer.
COS(expression)	This function is used to get the trigonometric cosine of an expression.
COSH(expression)	This function is used to get the hyperbolic cosine of an expression.
EXP(expression)	This function is used to get the Eulers number e raised to the power of x.
FLOOR(expression)	To get the value of an expression rounded down to the nearest integer.
LOG(expression)	To get the natural logarithm (base e) of an expression.
LOG10(expression)	To get the base 10 logarithm of an expression.
RANDOM()	To get a pseudo random number (type double) greater than or equal to 0.0 and less than 1.0.
ROUND(expression)	To get the value of an expression rounded to an integer (if the result type is float) or rounded to a long (if the result type is double).
SIN(expression)	To get the sine of an expression.
SINH(expression)	To get the hyperbolic sine of an expression.
SQRT(expression)	To get the positive square root of an expression.
TAN(expression)	To get the trigonometric tangent of an angle.
TANH(expression)	To get the hyperbolic tangent of an expression.

Fouad DAHAK
ESIA, 2024/2025

Section N° 5

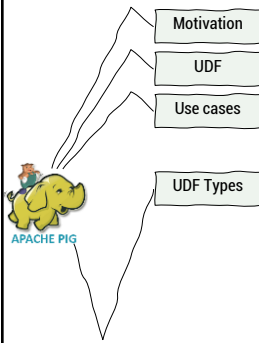
User Defined Functions

Fouad DAHAK, ENSIA, 2024/2025

User Defined Functions

Motivation

Section 05 40



Motivation	Pig has many built-in functions (e.g., SUM, LOWER), but sometimes they're not enough
UDF	A custom function written in Java to extend Pig Latin.
Use cases	Custom string processing (e.g., email normalization). Complex mathematical operations. Integrating third-party libraries (e.g., NLP, ML).
UDF Types	Filter Functions - The filter functions are used as conditions in filter statements. These functions accept a Pig value as input and return a Boolean value. Eval Functions - The Eval functions are used in FOREACH-GENERATE statements. These functions accept a Pig value as input and return a Pig result. Algebraic Functions - The Algebraic functions act on inner bags in a FOREACHGENERATE statement. These functions are used to perform full MapReduce operations on an inner bag.

Fouad DAHAK
ESIA, 2024/2025

User Defined Functions Section 01

Definition 41



```
package myudfs;

import java.io.IOException;
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;

public class ToLowerCase extends EvalFunc<String> {
    public String exec(Tuple input) throws IOException {
        if (input == null || input.size() == 0 || input.get(0) == null)
            return null;
        return input.get(0).toString().toLowerCase();
    }
}
```

Fouad DAHAK
ESIA, 2024/2025 **ensia**

User Defined Functions Section 01

Utilization 42



```
REGISTER 'myudfs.jar';
DEFINE ToLower myudfs.ToLowerCase();

data = LOAD '/shared/names.csv' USING PigStorage(',') AS (name:chararray);
lower_names = FOREACH data GENERATE ToLower(name);
DUMP lower_names;
```

Fouad DAHAK
ESIA, 2024/2025 **ensia**

Chapter : 43

Thank You

Q & A

Fouad DAHAK
ESIA, 2024/2025 **ensia**